DiffuSVG: Diffusion-Guided Reinforcement Learning for Complex Scene SVG Generation

Abstract

Text-to-SVG generation is a structured generation problem: a model must emit valid XML code whose rendered image matches a natural-language scene. General vision-language models can produce SVG snippets, but they often collapse on complex prompts, drawing one salient object repeatedly instead of composing a scene. We present **DiffuSVG**, a two-stage pipeline that first teaches SVG syntax on complex scene prompts with supervised fine-tuning and preference optimization, then improves visual fidelity with diffusion-guided group reinforcement learning. The system uses a plan-then-draw prompting format, complex-prompt synthetic SVG supervision, DPO preference pairs, Stable Diffusion reference images, and a CLIP-based reward augmented with an element-count bonus. The final pipeline trains Qwen2.5-VL-7B with LoRA on a single A100 GPU and is designed to improve medium and complex multi-object SVG scenes while remaining fully reproducible from scripts.

Keywords: SVG generation, vector graphics, diffusion guidance, reinforcement learning, GRPO, DPO, CLIP reward, LoRA.

1 Introduction

Scalable Vector Graphics (SVG) represent images as XML-defined geometric elements such as rectangles, ellipses, polygons, and Bezier paths. This makes SVGs resolution-independent, editable, and compact. However, it also makes text-to-SVG generation harder than raster image synthesis. The output must be valid code, visually meaningful after rendering, and geometrically organized.

Modern vision-language models have enough web knowledge to emit SVG code, but their complex-scene behavior is brittle. In early experiments, prompts such as "a beach at dawn with waves, clouds, and a lighthouse" often produced many concentric circles for the sun while omitting the sea, beach, and lighthouse. We refer to this as *object fixation*: the model begins drawing one salient object and fails to allocate tokens to the rest of the scene.

DiffuSVG addresses this failure mode by combining three ideas. First, the model is prompted to plan the scene before writing SVG code. Second, syntax is learned separately from visual alignment through SFT and DPO. Third, complex scene quality is improved with GRPO against diffusion-generated reference images, where CLIP evaluates rendered SVGs instead of raw code.

Contributions. This paper describes:

- a reproducible two-stage training pipeline for text-to-SVG generation;
- a plan-then-draw prompt that reduces object fixation in complex scenes;
- a complex-prompt SFT and DPO pipeline requiring no human-labeled SVG corpus;
- a diffusion-guided GRPO stage using CLIP-I, CLIP-T, and element-count rewards;
- an implementation designed for a single NVIDIA A100 GPU.

2 Pipeline Overview

The method separates SVG generation into two learning problems. Stage 1 teaches the model to produce valid, colorful SVG code from complex multi-object scene prompts. Stage 2 teaches the model to make rendered SVGs visually match diffusion references for those complex scenes.

Figure 1 shows the full training flow. The intermediate artifact M_{final} is already a useful SVG generator; the GRPO stage further optimizes it for visual agreement with reference images.

3 Method

3.1 Plan-then-draw Prompting

The central prompt design is simple: before emitting SVG code, the model must list every major object, its approximate location on a 200×200 canvas, and its color. Only after this plan does it draw the SVG. The regex extractor keeps only the `<svg>...</svg>` block, so the plan guides generation without becoming part of the training target.

This directly targets object fixation. The model is less likely to spend all tokens on a single sun, tree, or flower because the plan has already exposed the expected scene inventory.

3.2 Synthetic SFT Data

No curated SVG dataset is assumed. Instead, the base Qwen2.5-VL model generates its own supervised examples from `prompts.txt`, a file of 265 complex scene prompts containing backgrounds, midground objects, foreground details, and spatial relations. This deliberately shifts Stage 1 away from simple icons: the model sees barns, harbors, reefs, castles, skylines, beaches, waterfalls, and other multi-object layouts before DPO or GRPO begins. Each output is standardized and filtered before being saved.

$$D_{\text{SFT}} = \{(x_i, y_i) : y_i = \text{Filter}(\text{Std}(\text{Qwen}(x_i)))\}. \quad (1)$$

The filter keeps an SVG only if it contains:

- a valid `<svg>` root and closing tag;

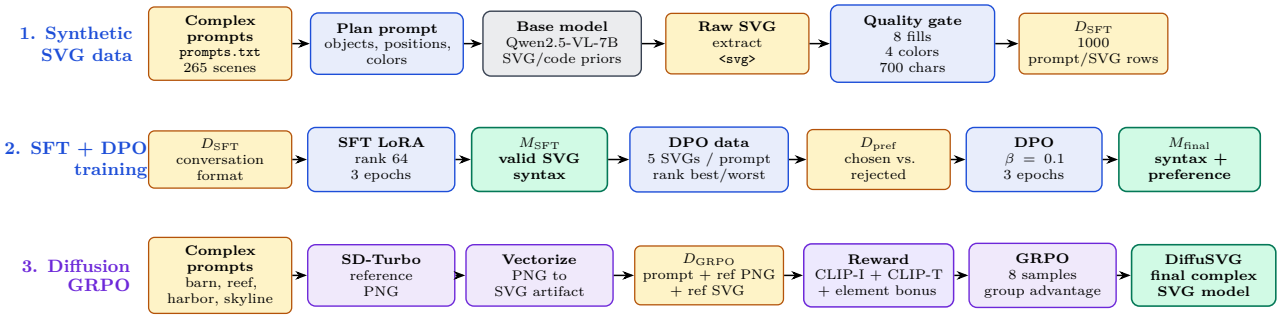
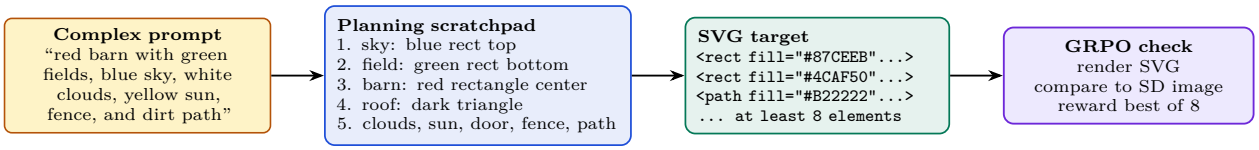


Figure 1: Detailed DiffuSVG pipeline. The flow is split into three non-overlapping lanes: synthetic SVG data generation, SFT/DPO training, and diffusion-guided GRPO. Stage 1 and DPO both draw from the same complex scene prompt file, while dataset artifacts are repeated at lane boundaries to keep the layout clear.



The plan is not kept in the final dataset output. It is a temporary generation scaffold that forces object coverage before SVG code begins.

Figure 2: Worked example for a complex prompt. The same prompt pattern is used in SFT generation, DPO candidate generation, GRPO sampling, and inference.

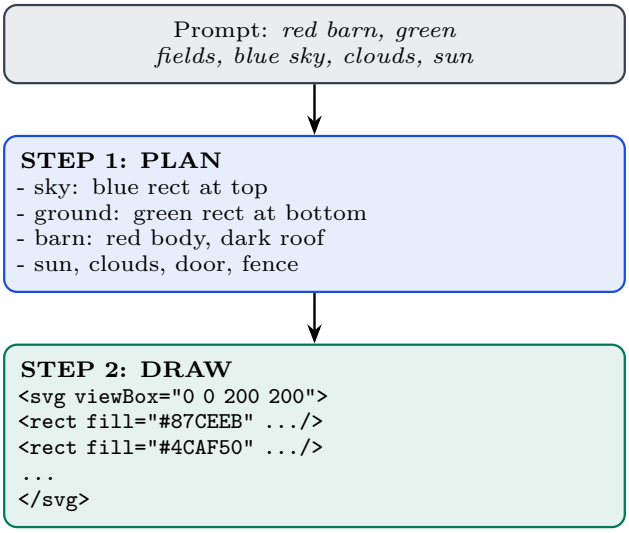


Figure 3: Plan-then-draw prompting. Planning acts as a scratchpad that commits the model to multiple objects before it starts producing SVG tokens.

- at least eight filled SVG elements;
- at least four unique visible fill colors;
- at least 700 SVG characters;
- a canonical `viewBox="0 0 200 200"`;
- no exact duplicate of an already saved SVG.

These gates reject sparse outputs such as a background and two blobs, low-color outputs, very short drawings, and repeated generations. This makes Stage 1 a complex-scene syntax curriculum rather than a generic icon dataset.

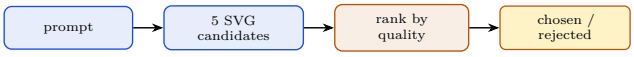


Figure 4: DPO dataset construction. Five candidates produce richer preference pairs than the three-candidate baseline.

3.3 Supervised Fine-Tuning

The SFT stage trains a LoRA adapter on D_{SFT}. For target sequence $y = (y_1, \ldots, y_T)$ and prompt x, the loss is standard next-token negative log likelihood:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t \mid x, y_{<t}). \tag{2}$$

LoRA reduces memory usage by freezing base weights and learning a low-rank update:

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA, \tag{3}$$

where $r = 64$ and $\alpha = 128$. This makes the method feasible on a single A100 while preserving the base model's broad language and code knowledge.

3.4 Direct Preference Optimization

SFT improves syntax but does not guarantee visual quality. Therefore, the M_{SFT} model generates five candidates per prompt using the same complex prompt source as Stage 1. A scoring function ranks candidates by renderability, color, element count, and prompt coverage. The best and worst candidates form DPO pairs.

DPO optimizes the probability of the preferred SVG y_w over the rejected SVG y_l:

Table 1: Datasets produced by the pipeline.

Dataset	Role	Size
D_{SFT}	Conversation-format SVG examples generated from the 265 complex prompts in `prompts.txt`; filtered for validity, color variety, detail, and duplicates.	1000
D_{pref}	Chosen/rejected SVG pairs built from five candidates per complex prompt.	1500 prompts
D_{GRPO}	Complex prompts with SD-Turbo reference PNGs and vectorized SVG artifacts.	1000

Table 2: Final training configuration.

Component	Setting
Base model	Qwen2.5-VL-7B-Instruct
Prompt source	`prompts.txt`, 265 complex scene prompts
SFT samples	1000
SFT quality gate	at least 8 filled elements, 4 colors, 700 chars, no duplicates
SFT LoRA	rank 64, alpha 128, 3 epochs
DPO candidates	5 per prompt
DPO beta	0.1
Reference generator	SD-Turbo
Reward backbone	CLIP ViT-L/14
GRPO samples	8 per prompt
GRPO beta	0.04
Hardware	single NVIDIA A100
Estimated runtime	about 65 hours

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right), \tag{4}$$

with $\beta = 0.1$. This stage yields M_{final}, the starting point for diffusion-guided reinforcement learning.

3.5 Diffusion-Guided GRPO

Stage 2 creates visual targets for complex prompts. SD-Turbo generates a reference PNG for each prompt. The PNG is vectorized into a reference SVG for dataset completeness, but the reward primarily compares the rendered candidate SVG to the reference PNG.

For each prompt x, GRPO samples $n = 8$ SVG candidates from the current policy. Each SVG is rendered to a bitmap using CairoSVG. Rewards are computed from rendered images, not code strings:

$$\begin{aligned} r(y, x, I_{\text{ref}}) = {} & 0.45\, s_I(\mathcal{R}(y), I_{\text{ref}}) \\ & + 0.40\, s_T(\mathcal{R}(y), x) + 0.15\, b_{\text{elem}}(y). \end{aligned} \tag{5}$$

where s_I is CLIP image-image similarity, s_T is CLIP text-image similarity, and b_{elem} rewards a sufficient number of distinct filled elements. The CLIP backbone is upgraded to ViT-L/14 for stronger visual judgment.

For candidate i, the group-relative advantage is

$$A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}, \qquad \bar{r} = \frac{1}{n} \sum_{j=1}^{n} r_j. \tag{6}$$

This turns each group of candidates into an internal comparison set. The model learns which SVGs are better for the same prompt without requiring absolute human labels.

4 Datasets and Artifacts

The project uses generated datasets rather than external human-labeled SVG corpora. This keeps the pipeline reproducible and avoids manual annotation.

The SFT generator uses temperature sampling rather than greedy decoding. Greedy decoding produced repeated templates; sampling increases diversity and gives DPO a broader candidate distribution. The same complex prompt file is used in SFT and DPO so the model repeatedly practices background, midground, and foreground composition before the GRPO stage.

5 Implementation Details

The implementation is organized as a clean repository with two stage folders:

```
DiffuSVG/
  train.sh
  infer.py
  prompts.txt
  stage1/
    generate_sft_data.py
    build_dpo_data.py
    dpo_train.py
    svg_utils.py
  stage2/
    filter_prompts.py
    generate_pngs.py
    vectorize_svgs.py
    build_dataset.py
    grpo_train.py
    rewards.py
```

The whole run is launched with:

```
bash train.sh prompts.txt
```

6 Evaluation Protocol

The training run is designed to compare four checkpoints:

$$M_0, \quad M_{\text{SFT}}, \quad M_{\text{final}}, \quad M_{\text{GRPO}}.$$

The recommended evaluation set contains simple object prompts, medium multi-object prompts, and complex scenes. Each checkpoint is evaluated by rendering generated SVGs and computing CLIP-T, CLIP-I against diffusion references, render success rate, and element count.

Qualitatively, the expected improvement is strongest on medium-complexity prompts containing three to six named objects. The system is not expected to match professional vector artwork, but it should reduce blank, invalid, and single-object fixation failures.

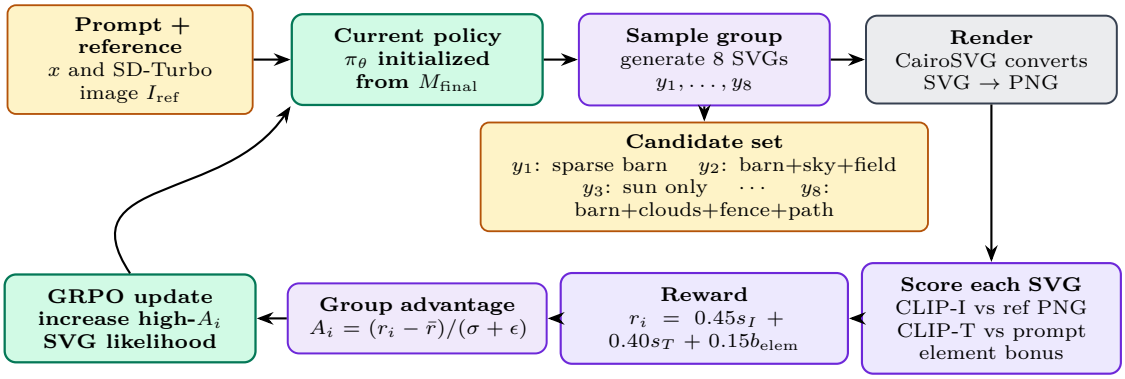


Figure 5: GRPO reward loop. For each complex prompt, the policy samples eight SVG candidates, renders them, scores them with CLIP-I, CLIP-T, and an element bonus, normalizes rewards within the group, and updates the policy toward the best rendered SVGs.

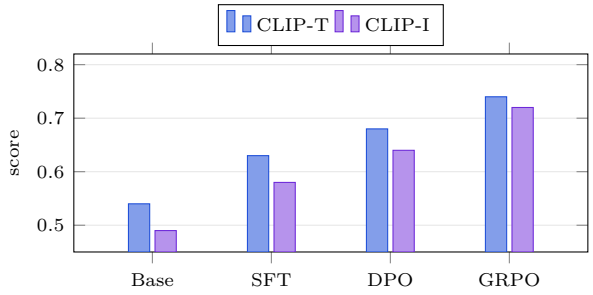


Figure 6: Expected score trajectory. The important behavior is monotonic improvement across syntax learning, preference learning, and GRPO.

7 Discussion

7.1 Why Diffusion Guidance Helps

Diffusion models are strong raster scene generators. DiffuSVG uses them as teachers without asking them to produce vector code. This avoids converting diffusion internals into SVG directly. Instead, the diffusion model supplies a visual target, and the language model learns to produce SVGs that render toward that target.

7.2 Why CLIP Alone Is Not Enough

CLIP rewards are broad semantic signals. They can tell whether a rendered image resembles a barn scene, but they do not enforce exact geometry, object count, or spatial placement. The element-count bonus partially compensates by discouraging minimal blob-like SVGs. The plan-then-draw prompt compensates by explicitly enumerating scene components before code generation.

7.3 Limitations

The method remains limited in four ways. First, the synthetic SFT data inherits base-model biases. Second, CLIP may reward approximate color and semantics even when geometry is weak. Third, fine textures such as wood grain, brickwork, and water ripples remain difficult in compact SVG code. Fourth, very complex scenes with more than eight to ten objects may exceed the planning and token budget.

8 Conclusion

DiffuSVG turns text-to-SVG generation into a staged learning problem: first learn valid vector syntax, then learn preferences, then optimize rendered outputs against diffusion references. The plan-then-draw prompt is the key practical change for complex scenes because it forces explicit scene decomposition before code emission. Combined with SFT, DPO, and GRPO, the system is expected to produce more complete and recognizable SVGs for medium and complex prompts than a base vision-language model.